

Robotic Arm Grasping + Multimodal Visual Understanding + SLAM Navigation

Robotic Arm Grasping + Multimodal Visual Understanding + SLAM Navigation

1. Course Content
2. Starting the Agent
3. Running the Cases
 - 3.1 Starting the Program
 - 3.2 Testing Cases

1. Course Content

- After running the example program, combine Nav2 navigation, robotic arm grasping, and AI large model visual understanding to perform complex task functions.
- **Note: This chapter requires prior configuration of the map mapping file.**

[!NOTE]

- The difference between the text version and the voice version lies in the instruction input method, and the text version does not require speech recognition and speech synthesis.

2. Starting the Agent

Note: All test cases must start the Docker agent first. If it is already started, there is no need to start it again.

Enter the following command in the vehicle terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:

```
jetson@yahboom: ~  
jetson@yahboom:~$ ./open_agent.sh  
[1749538760.063253] Info | TermiosAgentLinux.cpp | init | running... | fd: 3  
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4  
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81  
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0  
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)  
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)  
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)  
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)  
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)  
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)  
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)  
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)  
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)  
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)  
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)  
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)  
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)  
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)  
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)  
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)  
[1749538761.051660] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)  
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)  
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)  
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)  
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

3. Running the Cases

3.1 Starting the Program

Open the terminal on the vehicle and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

- Start the text interaction node in the terminal:

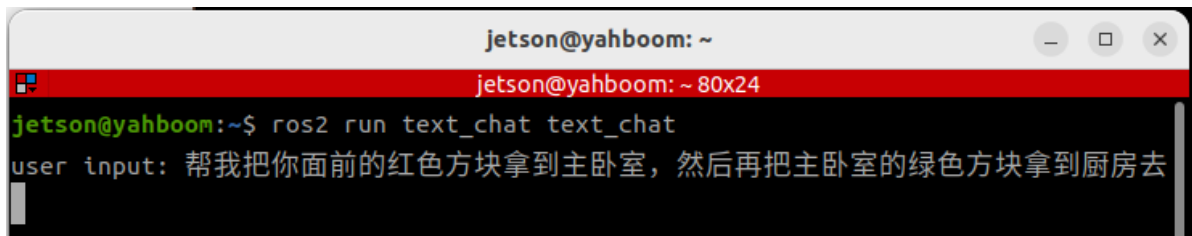
```
ros2 run text_chat text_chat
```

3.2 Testing Cases

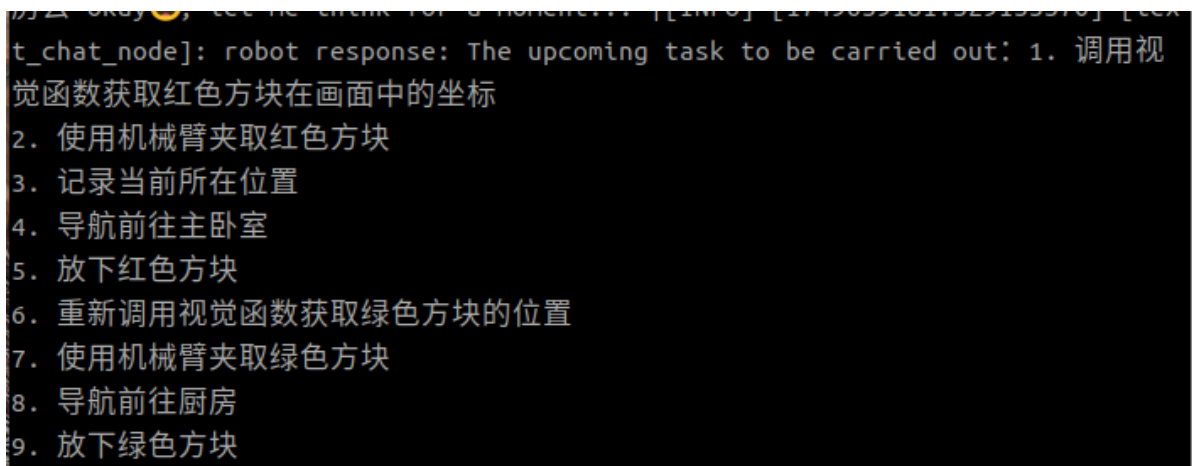
Here are two reference test cases; users can create their own test instructions.

- Please help me move the red block in front of you to the master bedroom, and then move the green block from the master bedroom to the kitchen.

Test case input in the text interaction terminal:



Task steps planned by the decision-making large language model:



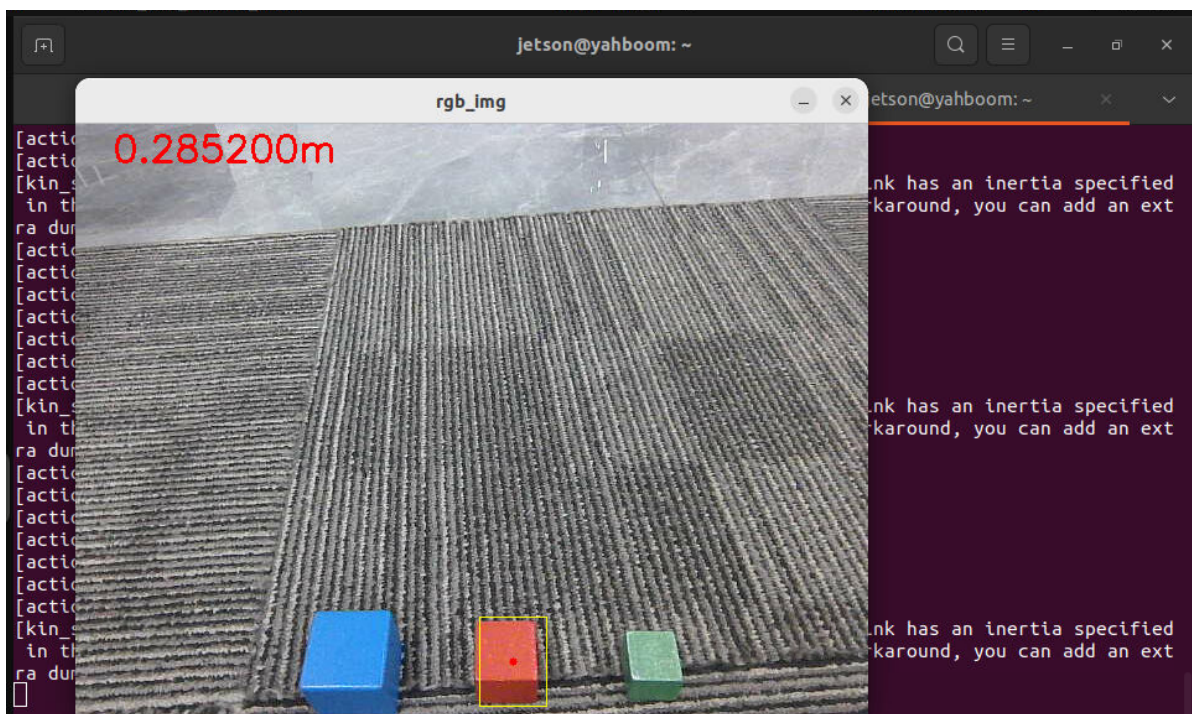
The execution layer large language model will then execute according to these task steps:

```

user input: [INFO] [1749639184.539887469] [text_chat_node]: robot response: "
action": ['seewhat()'], "response": 好的，我先看看面前的红色方块在哪里。
[INFO] [1749639190.143151310] [text_chat_node]: robot response: "action": ['g
rasp_obj(325, 287, 374, 346)'], "response": 我已经看到了红色方块的位置，现在
准备用机械臂夹取它。
[INFO] [1749639222.621630102] [text_chat_node]: robot response: "action": ['g
et_current_pose()'], "response": 红色方块已经夹取成功啦，接下来我需要记录一下
当前位置。
[INFO] [1749639226.216263475] [text_chat_node]: robot response: "action": ['n
avigation(A)'], "response": 当前位置已经记录好了，现在我将导航前往主卧室。
[INFO] [1749639262.719828763] [text_chat_node]: robot response: "action": ['p
utdown()'], "response": 我已经到达主卧室了，现在准备放下红色方块。
[INFO] [1749639273.046873263] [text_chat_node]: robot response: "action": ['s
eewhat()'], "response": 红色方块已经成功放下，接下来我需要看看主卧室里的绿色
方块在哪里。
[INFO] [1749639277.110579644] [text_chat_node]: robot response: "action": ['g
rasp_obj(358, 256, 404, 312)'], "response": 我已经看到了绿色方块的位置，现在
准备用机械臂夹取它。
[INFO] [1749639317.671182176] [text_chat_node]: robot response: "action": ['n
avigation(G)'], "response": 绿色方块已经夹取成功啦，现在我将导航前往厨房。
[INFO] [1749639342.209259442] [text_chat_node]: robot response: "action": ['p
utdown()'], "response": 我已经到达厨房了，现在准备放下绿色方块。
[INFO] [1749639352.627143524] [text_chat_node]: robot response: "action": ['f
inishtask()'], "response": 绿色方块已经成功放下，所有任务都完成了！还有什么需
要我帮忙的吗？

```

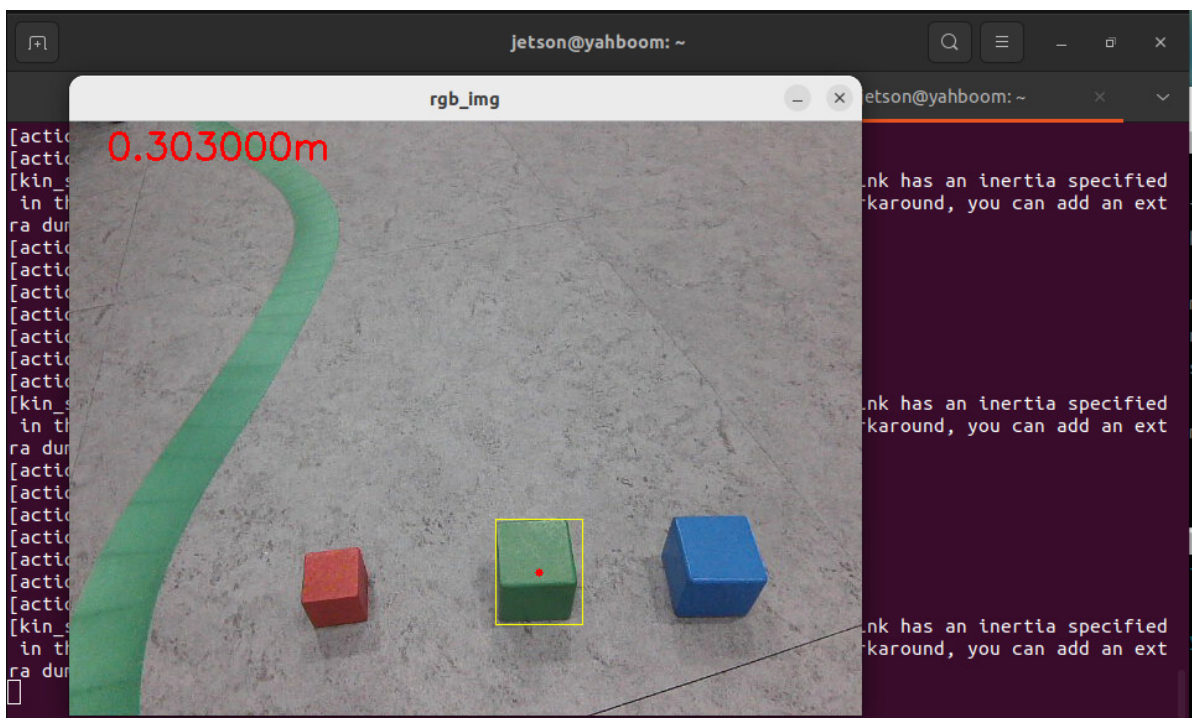
The robot will first grab the red block in front of it as instructed.



Then it navigates to the "bedroom" and uses the robotic arm to put down the red block.


```
jetson@yahboom: ~
ra dummy link to your URDF.
[kin_srv-2] [WARN] [1749623506.157489786] [kdl_parser]: The root link base_link has an inertia specified
in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an ext
ra dummy link to your URDF.
[kin_srv-2] [WARN] [1749623506.164689182] [kdl_parser]: The root link base_link has an inertia specified
in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an ext
ra dummy link to your URDF.
[kin_srv-2] [WARN] [1749623506.172139812] [kdl_parser]: The root link base_link has an inertia specified
in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an ext
ra dummy link to your URDF.
[kin_srv-2] [WARN] [1749623506.179279930] [kdl_parser]: The root link base_link has an inertia specified
in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an ext
ra dummy link to your URDF.
[model_service-3] [INFO] [1749623524.422449909] [model_service]: "action": ['get_current_pose()'], "resp
onse": 红色方块已经夹取成功, 接下来我记录一下当前位置。
[action_service-4] [INFO] [1749623533.693718769] [action_service]: Recorded Pose - Position: x=0.1, y=0.
1, z=0.0
[action_service-4] [INFO] [1749623533.695045453] [action_service]: Recorded Pose - Orientation: x=0.0, y
=0.0, z=0.0, w=1.0
[model_service-3] [INFO] [1749623550.587609008] [model_service]: "action": ['navigation(A)'], "response"
: 当前位置已经记录好了, 现在我将导航前往主卧室。
[action_service-4] [INFO] [1749623559.013915073] [action_service]: navigation Goal accepted
[action_service-4] [INFO] [1749623585.958945732] [action_service]: Navigation succeeded!
[model_service-3] [INFO] [1749623600.505944299] [model_service]: "action": ['putdown()'], "response": 我
已经到达主卧室, 现在将红色方块放下。
```

After observing, finding, and grasping the green block in the "master bedroom," it proceeds to the "kitchen."



After putting down the green block in the "kitchen," the robot indicates that the task is complete and enters a waiting state. At this point, enter "End current task, let the robot end the task" in the interaction terminal.